תרגול ממשקים מחלקת Drone

שם _____ כיתה _____ תאריך _____

המשימות מתוך Questions.cs בפרויקט .InterfacesVideo את הקוד כותבים בפרויקט המצורף.

משימה 1 מימוש מחלקת Drone

Drone הוא כלי תחבורה המסוגל לעוף. כדי להפעיל אותו צריך להטעין אותו.

א. השלימו את כותרת המחלקה לפי כללי ההורשה ומימוש הממשקים: בחרו את מחלקת הבסיס המתאימה מתוך הפרויקט, ואת שני ממשקי היכולות הקיימים שעל Drone לממש. אין ליצור ממשקים חדשים.

ב. הוסיפו שתי תכונות מתאימות:

- שם הדגם של הרחפן, מסוג .string

- אחוז הסוללה, מסוג .int הניחו שערכו בין 0 ל-100. בחרו שמות מתאימים לתכונות והגדירו בנאי שמקבל את שני הערכים. אפשר להשתמש בייצוג התכונות שלמדתם בכיתה; אין צורך ב-.protected

ג. ממשו את Fly לפי החתימה של הממשק:

- אם אחוז הסוללה נמוך מ-10, הדפיסו שהרחפן אינו יכול להמריא, בלי לשנות את אחוז הסוללה.

- אחרת, הדפיסו הודעת טיסה הכוללת את שם הדגם, והפחיתו 10 מאחוז הסוללה.

ד. ממשו את Charge לפי החתימה של הממשק: הגדירו את אחוז הסוללה ל-100 והדפיסו הודעת טעינה הכוללת את שם הדגם ואחוז הסוללה לאחר הטעינה.

שמרו את החתימות הקיימות: הפעולות אינן מקבלות פרמטרים ואינן מחזירות ערך. ההודעות ואחוז הסוללה מאפשרים לעקוב אחר הפעולות.

שלד המחלקה בקובץ Questions cs

```
public class Drone
{
}
```

משימה 2 שילוב הרחפן בקוד הקיים ובדיקת הרצה

לאחר השלמת משימה 1, פתחו את הפעולה Main בקובץ Program.cs.

א. צרו אובייקט אחד מטיפוס Drone עם שם דגם לבחירתכם וסוללה בערך 0. צרו אותו לפני הגדרת המערכים.

ב. הוסיפו את אותו אובייקט למערך devices מטיפוס []IChargeable, וגם למערך flyers מטיפוס []IFlyable. אין ליצור שני רחפנים שונים.

ג. הריצו את הפרויקט ובדקו:

- ChargeAll מפעילה גם את Charge של הרחפן ומטעינה אותו ל-100.

- לאחר מכן FlyAll מפעילה גם את Fly של אותו רחפן.

- לאחר הטיסה אחוז הסוללה של הרחפן הוא 90. הוסיפו ל-Main הדפסה של אחוז הסוללה לאחר FlyAll כדי לאמת זאת.

ד. ודאו שהרחפן משתלב בלי לשנות את גוף FlyAll או ChargeAll, ובלי להוסיף תנאי מיוחד הבודק אם האובייקט הוא Drone. כתבו כאן בהערה משפט המסביר כיצד הממשקים מאפשרים זאת.

המערכים הם אוסף הטיפוסים שביקשתם להפעיל; ההפעלה נעשית דרך הממשק, וכל אובייקט מבצע את המימוש של המחלקה שלו.

משימה 3 הרחבת חוזה הטעינה ומצב הסוללה

בצעו משימה זו לאחר השלמת משימות 1 ו-2.

א. פתחו את Interfaces.cs והוסיפו לממשק הקיים IChargeable את החתימה: string ChargingStatus(); הפעולה אינה מקבלת פרמטרים ומחזירה מחרוזת המתארת את מצב הסוללה. אין לכתוב גוף לפעולה בתוך הממשק בתרגול זה.

ב. לפני תיקון המחלקות, נסו לבנות את הפרויקט ובדקו את השגיאות. ElectricCar, Phone, RobotVacuum וגם Drone שמימשתם חתומות על החוזה. כעת כל אחת מהן חסרה מימוש ל-ChargingStatus. כתבו כאן בהערה: מדוע שינוי אחד בממשק דורש תיקון בכמה מחלקות?

ג. תקנו את כל ארבע המחלקות, לא רק את Drone:

- בכל מחלקה שמרו אחוז סוללה באמצעות תכונה מסוג int בטווח 0 עד 100. ב-Drone השתמשו בתכונה ממשימה 1; אין להוסיף תכונת סוללה כפולה.

- ב-ElectricCar, ב-Phone וב-RobotVacuum אתחלו את הסוללה ל-0. שמרו אפשרות ליצור אותן בבנאי ללא פרמטרים, כפי שנעשה ב-Main.

- עדכנו את Charge בכל מחלקה כך שתטעין את הסוללה ל-100, ותשמור גם הודעה מתאימה למכשיר.

- ממשו בכל מחלקה פעולה ציבורית עם החתימה string ChargingStatus(). הפעולה תחזיר את הטקסט המתאים; היא לא תדפיס ולא תשנה את הסוללה.

ד. השתמשו בכללים השונים הבאים, כולל ערכי הגבול:

מכשיר	**Empty**	**Low**	**Ready**
Drone	0-9	10-29	30-100
RobotVacuum	0-4	5-19	20-100
Phone	0-14	15-39	40-100
ElectricCar	0-19	20-49	50-100

כל האחוזים הם בטווח 0 עד 100. אין צורך בטיפול בחריגות.

Ready כאן הוא שם של קטגוריית מצב סוללה, ולא בדיקה אם המכשיר יכול לעוף.

ה. בדקו כל מחלקה באחוז 0, באחוז הסף הראשון פחות 1, באחוז הסף הראשון, באחוז הסף השני פחות 1, באחוז הסף השני, ובאחוז 100. לדוגמה, ב-Drone בדקו 100 ,30 ,29 ,10 ,9 ,0. בדקו גם ש-Charge משנה את הסוללה ל-100 ומצב הסוללה אחריה הוא "Ready". הגדירו ערכי בדיקה באמצעות התכונה או בנאי מתאים, לפי המימוש שלכם.

ו. הוסיפו ל-Main לולאה על devices מטיפוס IChargeable[]. בכל איטרציה קראו ל-ChargingStatus והדפיסו את המחרוזת שהוחזרה. עשו זאת גם לפני ChargeAll וגם לאחריה. אין לבדוק אם האובייקט הוא Phone ,Drone או סוג מסוים אחר. ודאו שהפרויקט חוזר להיבנות לאחר שכל המחלקות מקיימות את החוזה החדש.